

4. **F10** over, **F11** into, **F5** continue.

Append nine items and press F5 through the stops. On **Monday's version** it hits every append and the address changes every time: nine appends, nine addresses. On **today's version** it hits five times, capacity 1, 2, 4, 8, 16, and the grows get rarer as the array gets bigger. The array moves. Anything still holding the old address is holding a lie.

Close: Friday you use these same four moves on a program that misbehaves, and the address in that watch window is the whole story.

Beat 5. What this unlocks (6 min)

- **a05 TODO 5, `append`**. Written twice now. Read `setSize` in `Trace.cpp` first: same reallocate, copy, delete, repoint, worked out in full.
- **a05 TODO 6, the destructor**. One line, and beat 3 is the reason.
- That is §7 of the page. **§9 comes after it, and in order.**
- Friday: the copy that bites.

Worth a sentence if it comes up: `PyList` uses `int`, `Trace` uses `unsigned`, which is why a05's bounds check is `i >= mySize` with no `i < 0`. `getIndex` has no bounds check and a05 TODO 4 wants one that throws `range_error`. The `Makefile` header still says "Makefile for the Vec class".

CS112: Wednesday 30 September, 12:15 to 1:20

Monday we wrote `PyList` together. A class holding `Item *myArray`, `mySize` and `myCapacity`; a constructor that sets the pointer to `nullptr`; an `append` that makes room for one more item every time it is called. It compiles, it passes its test, and every line in it is correct.

Today: the object is not the data. `myArray` holds an address, the items sit somewhere else, and that address moves whenever the array grows. Once that is visible, two things follow at once. Growing by one costs 4,950 copies where 127 would do, and nothing ever gives the memory back.

Their <code>append</code>	100 appends = 4,950 copies, 100 allocations
Doubling	the same 100 appends = 127 copies, 8 allocations
No destructor	AddressSanitizer: 20 bytes leaked in 1 allocation

TIME	BEAT
12:15	Daily quiz
12:20	1. Their code works. What does it cost?
12:32	2. The fix, and <code>?</code> :
12:48	3. It leaks right now
12:58	4. The debugger, live, on their own class
13:14	5. What this unlocks

Everything below is `PyList`; the deck's `Vec` is the same class under another name. If the clock slips, shorten beat 1. Beat 4 is the one nothing else replaces.

Beat 1. Their code works. What does it cost? (12 min)

Open `PyList.cpp`. Run `make test`. It passes. Say plainly that this is correct code and nobody is about to tell them otherwise. Then:

Every append allocates a new array and copies everything across. How many copies is that, for a hundred appends?

Let them work at it: $1 + 2 + \dots + 99$. Then the target, no code yet.

```
grow by one : 4950 item copies, 100 allocations
doubling   : 127 item copies,   8 allocations
```

Same result, same class, same tests. Thirty-nine times fewer copies.

Beat 2. The fix, and `?:` (16 min)

```
void PyList::append(Item it) {
    if (mySize == myCapacity) {        // full; empty is full too
        int cap = (myCapacity == 0) ? 1 : myCapacity * 2;
        Item *bigger = new Item[cap];
        for (int i = 0; i < mySize; ++i)
            bigger[i] = myArray[i];
        delete [] myArray;
        myArray = bigger;
        myCapacity = cap;
    }
    myArray[mySize] = it;
    ++mySize;
}
```

Their whole `if (mySize == 0) ... else ...` is gone. Say it out loud: **the special case was never a special case**, it was an empty array that was full.

Write the version they can already read, then the one line beside it. Read it aloud as **"if this, then that, otherwise the other"**.

```
int cap;
if (myCapacity == 0) cap = 1;
else                 cap = myCapacity * 2;
```

`if` is a **statement**: it *does* something. `?:` is an **expression**: it *is* a value. That is why it can sit on the right of an `=` and the `if` cannot.

Three rules: choose **between two values for one variable**, never nest them, no side effects inside. Then the trap, worth running:

```
cout << x > 0 ? "positive" : "negative";
```

```
error: no match for 'operator>' (operand types are
      'std::basic_ostream<char>' and 'int')
```

`<<` binds tighter, so the compiler read `(cout << x) > 0`. Parenthesise it whenever it sits inside another expression.

Re-run Monday's test. It still passes: after two appends capacity is still 2, because doubling goes 0, 1, 2. **Their test cannot tell the two versions apart.** Thirty seconds on what a test does and does not prove.

Beat 3. It leaks right now (10 min)

Ask what happens to the array when a `PyList` goes out of scope. Then show them.

```
g++ -std=c++17 -g -fsanitize=address -o leak leak.cpp PyList.cpp
&& ./leak
```

SUMMARY: AddressSanitizer: 20 byte(s) leaked in 1 allocation(s).

```
PyList::~PyList() { delete [] myArray; myArray = nullptr; }
```

The compiler writes a destructor for you and it destroys the **members**. It destroys the pointer. It does not touch what the pointer points at.

If your class calls `new`, your class defines a destructor.

Re-run the sanitizer. Clean. That is the whole beat.

Beat 4. The debugger, live, on their own class (16 min)

They have never watched anyone drive it: `course-map.yml` has it arriving in the wk2 lab and "introduced inside a03 itself", so they only met it in writing. a05 §9 has them breakpoint `delete[]` and read an address out of Variables. This is the last class before they need it. Use `PyList`, not a05's `experiment`, which they should meet cold.

1. **Breakpoint.** Click the gutter left of the line number, red dot, on the `int cap = ...` line.
2. **F5**, pick the configuration. It stops on the dot.
3. **Variables.** Expand `this` for `myArray`, `mySize`, `myCapacity`. Add `myArray` to **Watch**.